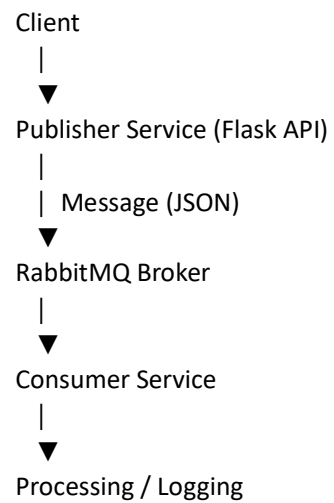


SOA Lab Exercise 2 – End-to-End Implementation

Objective

Demonstrate **loose coupling** using **asynchronous messaging** between services.

Architecture:



Step 1 — Install RabbitMQ

Download

<https://www.rabbitmq.com/download.html>

Install:

1. Erlang
2. RabbitMQ Server

Step 2 — Install Python Library

In VS Code terminal:

`pip install pika flask`

pika → RabbitMQ Python client

Step 3 — Create Project Folder

soa-lab-2/

|

└─ publisher.py

└─ consumer.py

Step 4 — Create Publisher Service (Flask)

Create **publisher.py**

```
from flask import Flask, request, jsonify
import pika
import json

app = Flask(__name__)

# RabbitMQ connection
connection = pika.BlockingConnection(
    pika.ConnectionParameters('localhost')
)

channel = connection.channel()

# Create queue
channel.queue_declare(queue='orders')

@app.route("/order", methods=["POST"])
```

```

def create_order():

    data = request.get_json()

    message = json.dumps(data)

    # Send message to RabbitMQ
    channel.basic_publish(
        exchange='',
        routing_key='orders',
        body=message
    )

    return jsonify({
        "status": "Order sent to queue",
        "data": data
    })

if __name__ == "__main__":
    app.run(port=5000)

```

Step 5 — Create Consumer Service

Create **consumer.py**

```

import pika
import json

connection = pika.BlockingConnection(
    pika.ConnectionParameters('localhost')
)

channel = connection.channel()

channel.queue_declare(queue='orders')

def process_order(ch, method, properties, body):

    order = json.loads(body)

    print("\nNew Order Received")
    print("-----")
    print("Product:", order["product"])
    print("Quantity:", order["quantity"])

```

```
channel.basic_consume(  
    queue='orders',  
    on_message_callback=process_order,  
    auto_ack=True  
)  
  
print("Consumer waiting for messages...")  
  
channel.start_consuming()
```

Step 6 — Run the Consumer

Open terminal 1:

```
python consumer.py
```

Output:

Consumer waiting for messages...

Consumer is now **subscribed to RabbitMQ queue**.

Step 7 — Run the Publisher

Open terminal 2:

```
python publisher.py
```

Flask will start:

```
http://127.0.0.1:5000
```

Step 8 — Send an Event

In PowerShell:

```
Invoke-RestMethod `  
-Uri http://127.0.0.1:5000/order `  
-Method POST `  
-ContentType "application/json" `  
-Body '{"product":"Laptop","quantity":2}'
```

Step 9 — Observe Consumer

Terminal 1 shows:

New Order Received

Product: Laptop

Quantity: 2

This proves:

- ✓ Publisher sent message
- ✓ RabbitMQ handled delivery
- ✓ Consumer processed event

Step 10 — Demonstrate Loose Coupling

Stop consumer.

Send order again.

Publisher still works because:

Publisher → Broker → Queue

Consumer can come online later and process messages.

This is **asynchronous messaging**.